

FPL League Platform Blueprint

End-to-End Implementation Plan & Architecture Guide

This document outlines the step-by-step technical plan to build a bespoke Fantasy Premier League tracking and analytics platform. The project is divided into logical, sequential phases designed to establish a robust foundation before moving into complex visualizations and forecasting mechanics.

Phase 1: Environment & Infrastructure

The goal of this phase is to establish the version control strategy and deploy the foundational cloud resources using a modern, declarative approach. This ensures the environment is reproducible and clean.

Step 1.1: Initialize the Modular Monorepo

- Create the root repository (`fpl-league-platform`).
- Establish the four core directories: `frontend/`, `backend/`, `data_pipeline/`, and `infrastructure/`.
- Set up base `.gitignore` rules for Python, Node.js, and Terraform.

Step 1.2: Infrastructure as Code (Terraform)

- Navigate to the `infrastructure/` directory.
- Write Terraform modules to provision a Google Cloud SQL instance (PostgreSQL) for transactional storage.
- Configure Cloud Run services for the API and frontend containers.
- Provision a Cloud Scheduler job and a lightweight Cloud Function for the polling mechanism.
- Apply the Terraform state to spin up the Development/MVP environment.

Phase 2: Data Modeling & ETL Pipeline

With infrastructure in place, the next objective is securely fetching data from the FPL endpoints, calculating custom metrics like rolling averages, and storing it relationally.

Step 2.1: PostgreSQL Schema Design

- Design tables for `manager`, `gameweek_metadata`, `manager_gameweek_scores`, and `transfers`.
- Implement SQLAlchemy models in the `data_pipeline/` directory.
- Run Alembic migrations to instantiate the schema in the Cloud SQL database.

Step 2.2: The Smart Polling Mechanism

- Write a lightweight Python Cloud Function that queries `/bootstrap-static/`.
- Implement the logic gate: check if the current gameweek has `finished == True` AND `data_checked == True`.
- If true, verify against the `gameweek_metadata` table to ensure it hasn't been processed yet.

Step 2.3: Batch Processing Engine

- Build the Python ETL worker triggered by the polling mechanism.
- Fetch league-specific data (manager picks, transfers).
- Calculate the **rolling 3-game average** for each manager using pandas or direct SQL window functions.
- Upsert the processed records into the PostgreSQL database.

Phase 3: The API Layer

This phase exposes the processed database records to the frontend via a high-performance REST API.

Step 3.1: FastAPI Core Setup

- Initialize the FastAPI application in the `backend/` directory.
- Configure CORS to allow requests from the future Next.js domain.
- Set up the database connection pool securely using GCP secret manager or environment variables.

Step 3.2: Endpoint Development

- GET /league/standings: Returns the current league table with point totals.
- GET /league/metrics: Returns the rolling 3-game average and form data.
- GET /league/transfers: Returns a chronological list of transfers made in the most recent gameweek (for the News Feed).

Phase 4: Frontend MVP (Mobile-Responsive UI)

Construct the public-facing portal where league managers will consume the data. Focus on clean, server-side rendered delivery.

Step 4.1: Next.js Foundation

- Initialize the Next.js application in the frontend/ directory.
- Configure Tailwind CSS for rapid, mobile-first styling.

Step 4.2: Component Assembly

- **League Table Component:** Display standard standings alongside the rolling 3-game average column.
- **Transfer News Feed:** Create a social-media style card layout highlighting who managers brought in and transferred out.
- Ensure the layout shifts gracefully from desktop grids to mobile stacks using Tailwind breakpoints.

Phase 5: Advanced Visuals & Machine Learning (Post-MVP)

Once the foundation is stable, you can introduce complex features that significantly elevate the platform's value.

Step 5.1: Dynamic Bar Chart Race

- Integrate D3.js or a library like react-chartjs-2.
- Create an endpoint GET /league/historical-ranks to feed week-by-week cumulative scores.
- Render the animated race component to visualize positional fluctuations over the season.

Step 5.2: Expected Points Forecasting Engine

- Extend the `data_pipeline` to ingest player-level underlying stats (`xG`, `xA`, `FDR`).
- Implement a machine learning model (e.g., `LightGBM`) to generate expected points (`xPts`) based on recent form and historical data.
- Aggregate player predictions into manager-level forecasts to generate automated "Head-to-Head Win Probability" match previews.

Strategic Note on Expansion: By maintaining a strict boundary between your ETL pipelines, REST API, and frontend applications within the monorepo, swapping out heuristics for advanced predictive models in Phase 5 will require zero changes to your UI components.